Predicting MBTA Rail On-Time Performance Using Distributed Machine Learning

Nirmalkumar Thirupallikrishnan Kesavan Pradnyesh Choudhari Raveendra Sanapala
EECE 5645: Parallel Processing for Data Analytics
Northeastern University
Spring 2026

1 Problem and Motivation

The MBTA operates seven rail routes (Red, Blue, Orange, and four Green Line branches) serving roughly 300,000 daily riders. On-time performance (OTP), the fraction of scheduled trips completed within a published tolerance, is the agency's primary public reliability metric, and being able to predict it ahead of a service day has real operational value for both riders and operators.

The data required to build such a model is large and heterogeneous. Five years of operational records across train events, passenger demand, service alerts, weather, and reliability measurements amount to over 150 million raw records spanning multiple sources with different schemas, granularities, and update frequencies. Processing and learning from this data efficiently requires a distributed approach.

This project builds a distributed machine learning pipeline on Apache Spark to predict daily OTP and to study how much parallelism concretely improves training time. We train four regression models under cross-validation and benchmark runtime as the number of parallel workers grows, evaluating both the computational gains and the predictive accuracy of the resulting models.

2 Dataset and Preprocessing

We draw from five data sources, all obtained from the MBTA Open Data portal and NOAA.

The reliability dataset contains 1 million rows of daily OTP numerator and denominator broken down by route, mode, and peak/off-peak indicator. This is the source of the target variable: we compute OTP as the ratio of on-time trips to total scheduled trips, giving a continuous value between 0 and 1. We restrict all modelling to Rail mode, which covers the seven routes of interest.

The train events dataset is the largest and most complex source at 151 million rows. Each row records a single arrival or departure event for a train at a stop, with a timestamp, route, direction, and stop identifier. We do not use these events directly as features; instead we derive headway statistics from them. Headway — the time between consecutive train arrivals at the same stop in the same direction — is a direct measure of service frequency and one of the strongest predictors of reliability. We aggregate daily per-route headway means and maximums, split by peak and off-peak periods, giving four headway features in the final dataset.

The gated entries dataset records fare-gate tap counts at 131 stations in 30-minute windows, totalling around 65 million rows over five years. These counts are aggregated to daily per-route totals split by peak and

off-peak periods, giving two demand features that capture how heavily a route is being used on a given day.

The service alerts dataset contains around 3.8 million records of disruptions, each tagged with a severity score from 1 to 10. A single number per day is not enough to capture the full picture, so we compute three severity indices. The tiered index is a weighted count that assigns multipliers of 1, 2, and 3 to low, medium, and high severity alerts respectively:

$$I_{\text{tiered}} = \text{count}(\text{sev } 1\text{–}4) \times 1 + \text{count}(\text{sev } 5\text{–}7) \times 2 + \text{count}(\text{sev } 8\text{–}10) \times 3$$

The quadratic index sums squared severities, which heavily penalises days with a small number of catastrophic events:

$$I_{\text{sq}} = \sum_i \text{sev}_i^2 \ / \ 10$$

The Disruption Burden Score multiplies total severity by a log-compressed alert count, rewarding both volume and intensity:

$$I_{\text{DBS}} = \sum_i \text{sev}_i \cdot \log(1 + \text{count})$$

Keeping all three allows the model to select whichever representation is most predictive.

The weather dataset is the smallest source at 2,269 rows, one per calendar day, from the NOAA GHCN station at Boston Logan Airport. It contributes five features: average, minimum, and maximum temperature, precipitation, and snowfall. Weather is broadcast identically to all routes on a given day since the station covers the whole network.

Each source is cleaned independently and stored as a Parquet table before joining. The train events files have three different schemas across years and require projecting onto a common column set before unioning. The gated entry files have inconsistent column ordering between pre-2024 and post-2024 exports, which is corrected by renaming columns explicitly before the union. Alert descriptions contain embedded newlines that inflate the apparent row count from 3.8 million to 34 million under standard CSV parsing; enabling multiline mode in the reader brings this back to the correct count. Headways are computed using a Spark window function over consecutive arrivals at the same stop, with implausibly short values (duplicate-logged events) and long values (end-of-service gaps) filtered out before aggregation.

The five cleaned tables are joined on (service date, route) to produce a master table of 19,229 rows with 20 columns and no missing values. Table 1 summarises the key statistics.

Table 1: Master table statistics.

Statistic	Value	
Total rows	19,229	
Training rows	17,469	2020–2023
Test rows	1,760	January–May 2024
Columns	20	
Null values	0	after fill
OTP mean (std)	0.849 (0.086)	
Per-route OTP range	0.775–0.951	Green-D to Blue

3 Feature Engineering and Models

The master table contains 20 columns drawn from the five source datasets. From the reliability data we have the target variable, OTP ratio, along with route identity and the peak/off-peak indicator. From the weather dataset we have five daily measurements: average, minimum, and maximum temperature, precipitation, and snowfall. From the gated entries dataset we have peak and off-peak daily tap totals per route. From the train events dataset we derive four headway statistics: mean and maximum headway split by peak and off-peak periods. From the service alerts dataset we have the three severity indices described in the previous section. The service date ties all of these together as the join key.

From these raw columns we construct the final feature set used for modelling. The service date is not usable directly, so we extract six temporal features from it: day of week, month, day of year, a weekend indicator, year, and a peak/off-peak binary. These capture calendar structure at different resolutions and are all known before the service day. Route identity is categorical and is one-hot encoded using a StringIndexer followed by a OneHotEncoder, both fit only on training data to prevent leakage into the test set. The remaining columns — weather, demand, headway, and alert features — are used as-is. All features are standardised to unit variance before training Linear Regression; tree-based models do not require this. The final feature vector has 28 dimensions: 5 weather, 2 demand, 4 headway, 3 alert, 6 temporal, and 8 route.

With the feature set defined, we train four regression models from PySpark MLlib. We choose these four because they represent a progression from simple to complex and cover the main families of supervised learning methods: a linear model, a single tree, a bagging ensemble, and a boosting ensemble.

Linear Regression. L2-regularised ($\lambda = 0.01$), optimised via L-BFGS. A global linear model that scales smoothly with input values, which makes it relatively robust when test inputs differ from the training distribution.

Decision Tree. Single tree with max depth 10. Fast to train but prone to high variance; it serves as a non-linear baseline against which the ensemble methods can be compared.

Random Forest. 100 trees with max depth 10, trained with bootstrap sampling and feature bagging at 1/3 of features per split. Averaging across many decorrelated trees reduces variance substantially compared to a single decision tree.

Gradient Boosted Trees (GBT). 50 boosting rounds with max depth 5 and learning rate $\eta = 0.1$. Each round fits a new tree to the residuals of the current ensemble, giving the update $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$. GBT is generally the strongest model on tabular data under IID conditions. The sequential dependency between rounds — where round m cannot begin until round $m - 1$ is complete — is also a key constraint when we evaluate parallelism in Section 5.

4 Results and Analysis

4.1 Evaluation Setup

We evaluate all models under two train/test split strategies. The primary split is time-based: we train on data from 2020–2023 and test on January–May 2024. This reflects the realistic forecasting scenario where the model is always predicting into a future period it has not seen. As a control, we also run a random 90/10 split where training and test rows are drawn from the same distribution by construction. This lets us isolate whether any differences in model performance are due to the models themselves or due to the nature of the temporal split. We report three metrics across both experiments: RMSE, MAE, and R^2. RMSE and MAE

measure prediction error in the same units as OTP; R^2 measures how much variance the model explains relative to a naive mean predictor.

4.2 Model Performance

Table 2 shows results under the time-based split. Linear Regression wins by a clear margin at $R^2 = 0.540$, which is the opposite of what one would normally expect. GBT and Random Forest, which are generally the strongest models on tabular data, achieve only $R^2 = 0.455$ and 0.462 respectively. Decision Tree performs worst at $R^2 = 0.255$, a result consistent with its known tendency to overfit. The train-to-test RMSE gap is also telling: LR's gap is only 0.011, while the tree models have gaps of 0.037–0.049. Ensemble methods are specifically designed to resist overfitting, so this magnitude of degradation suggests something beyond ordinary overfitting is at work.

Table 3 shows results under the random split, where the conventional ordering is fully restored. GBT leads at $R^2 = 0.833$, Random Forest follows at 0.826, and Linear Regression finishes last at 0.632. The R^2 recovered by switching from the time-based to the random split is $+0.378$ for GBT, $+0.364$ for Random Forest, $+0.535$ for Decision Tree, and only $+0.092$ for Linear Regression. This asymmetry is the key finding: tree models are severely hurt by the temporal split while LR is largely unaffected.

The explanation is temporal distribution shift. The test window covers only January through May, so temperatures are about 20% lower than the all-year training average. Gated entry counts are 11–17% higher in 2024 due to post-pandemic ridership recovery. The quadratic alert severity index drops 62% in the test period because the training set contains the 2022 Orange Line shutdowns, which have no equivalent in early 2024. Tree models are piecewise-constant and cannot extrapolate beyond the feature boundaries they learned from training data; when the test distribution shifts, entire subtrees become inactive and the model defaults to miscalibrated leaf predictions. Linear Regression handles the same shifts gracefully because its response is a continuous global function of the inputs; a systematic change in any feature produces a proportional, bounded change in the prediction.

Table 2: Test results under the time-based split.

Model	Train Time	RMSE	MAE	R^2
Linear Regression	**5.8 s**	**0.0628**	**0.0466**	**0.540**
Random Forest	33.7 s	0.0678	0.0508	0.462
GBT	91.1 s	0.0683	0.0519	0.455
Decision Tree	3.6 s	0.0798	0.0561	0.255

Table 3: Test results under a random 90/10 split.

Model	Train Time	RMSE	MAE	R^2
GBT	**91.5 s**	**0.0352**	**0.0248**	**0.833**
Random Forest	28.1 s	0.0359	0.0251	0.826
Decision Tree	2.0 s	0.0395	0.0265	0.790
Linear Regression	0.7 s	0.0522	0.0369	0.632

4.3 Hybrid Architecture

Given that tree models fail under temporal shift, we ask whether the problem can be addressed architecturally. We design a hybrid that uses Linear Regression as a stable baseline and adds a GBT trained on its residuals, controlled by a mixing parameter α:

$$\hat{y}(x) = f_{\text{LR}}(x) + \alpha \cdot f_{\text{GBT}}^{\text{resid}}(x), \qquad \alpha \in [0, 1.5]$$

Setting $\alpha = 0$ recovers pure LR; $\alpha = 1$ applies the full residual correction. The residual GBT is intentionally weakened (max depth 3, 30 rounds, learning rate 0.05) and is denied the three features that showed the largest distributional shift between training and test. Table 4 shows the α sweep under the time-based split. RMSE increases monotonically from $\alpha = 0$, meaning every positive contribution from the residual GBT makes performance worse. The optimal model is pure LR. This confirms that the tree-model failure under distribution shift cannot be fixed by composition — even a carefully restricted residual GBT encodes training-specific structure that does not transfer to the test period.

Table 4: α-sweep for the LR + residual-GBT hybrid (time-based split).

α	RMSE	R^2
0.0	**0.0628**	**0.540**
0.1	0.0631	0.535
0.3	0.0638	0.524
0.5	0.0647	0.510
1.0	0.0679	0.461
1.5	0.0720	0.394

4.4 Lag Feature Mitigation

Both the drift analysis and the hybrid experiment point in the same direction: tree-based models fail not because they are bad models, but because they lack *temporal context*. Their feature vectors describe only the current day, so when the 2024 test window has drifted weather, ridership, and alert distributions, the trees have no anchor to keep their predictions calibrated. Linear Regression survives because a smooth global hyperplane does not depend on any particular feature landing in a previously-seen range; tree leaves do.

This suggests a targeted intervention: add features that encode each route's recent OTP history directly, giving all four models a temporal anchor that drifts with time instead of being fixed to a training-era slice. We compute three backward-looking rolling averages of daily OTP per route — `otp_lag_7d`, `otp_lag_30d`, `otp_lag_90d` — plus an observation count `otp_lag_count_7d`. Critically, each row's lag features only consider *strictly prior* days (Spark's `rangeBetween(unboundedPreceding, -1)`), so today's OTP is never used to predict today. Cold-start rows at the very start of 2020 are filled with the training-period global mean to avoid nulls. This gives a 32-dimensional feature vector.

We retrain all four models with the new features, using identical hyperparameters to the baselines in Table **??**. Table 5 shows the side-by-side comparison on the primary time-based split.

Table 5: Impact of lag features on the time-based split. Every model improves substantially, with R^2 gains between $+0.146$ and $+0.176$.

	Original (28 feats)		With lag feats (32)		Change	
Model	RMSE	R^2	RMSE	R^2	Δ RMSE	ΔR^2
Linear Regression	0.0628	0.540	**0.0518**	**0.686**	-0.0109	$+0.146$
Decision Tree	0.0798	0.255	0.0698	0.431	-0.0101	$+0.176$
Random Forest	0.0678	0.462	0.0557	0.638	-0.0122	$+0.176$
GBT	0.0683	0.455	0.0570	0.620	-0.0113	$+0.165$

Every model gains between $+0.146$ and $+0.176$ in R^2, with RMSE dropping by roughly 16–17% across the board. To put this in context, the gap between each tree model's time-split R^2 and its random-split R^2 was previously around 0.36–0.54 (Table 5); lag features close approximately half of this drift gap in a single intervention.

The mechanism is intuitive. A tree on the original feature set can only split on things like "is temperature below 45?" or "is peak headway above 600 seconds?" — rules whose meaning shifts when the underlying distribution shifts. Once `otp_lag_30d` is available, the tree can split on "is this route's recent average OTP above 0.85?", which is self-calibrating: the threshold is always evaluated against the route's own recent state, not against a fixed training-era distribution. Linear Regression also improves because the lag features are highly correlated with the target and let LR explain additional variance that the original features could not.

A nuanced observation: even with lag features, LR still wins on the time-based split ($R^2 = 0.686$ vs GBT's 0.620). The tree models close most of the drift gap but do not pass LR. This tells us that some residual drift remains that lag features cannot fully absorb — the test window's weather and alert distributions are still different enough that tree splits over those features remain somewhat miscalibrated. LR's smooth extrapolation property continues to pay off on this residual. The practical lesson is that feature engineering and model choice are complementary defences against drift: lag features capture most of it, but the remainder still favours smooth models over piecewise-constant ones.

5 Parallelism Performance (Single-Node vs Multi-Node Compute)

5.1 Single-Node

We benchmark how training time changes as we increase the number of concurrent workers in cross-validation. Spark's CrossValidator accepts a parallelism parameter N that runs up to N fold evaluations at the same time. We test $N \in \{1, 2, 4\}$ across all four models and both evaluation splits.

We run the benchmark under two grid sizes to see whether having more work available affects the speedup. Stage 1 uses 2 parameter values with 3-fold cross-validation, giving 6 model trainings per CV run. Stage 2 doubles this to 4 parameter values, giving 12 trainings per run. In total we run 432 individual model trainings across both stages. All experiments run on a single-node cluster with 4 vCPUs and 16 GB RAM **`m5dxlarge`**.

5.1.1 Stage 1 Results

Figures 1 and 2 show wall-clock time and speedup curves for Stage 1 under the time-split and random-split respectively. The two splits produce nearly identical parallelism behaviour, which is expected — the split

method affects which rows go to train and test, but has no bearing on how Spark schedules concurrent model trainings. All models show modest speedups in the range of $1.1\times$ to $1.4\times$ at $N = 4$.

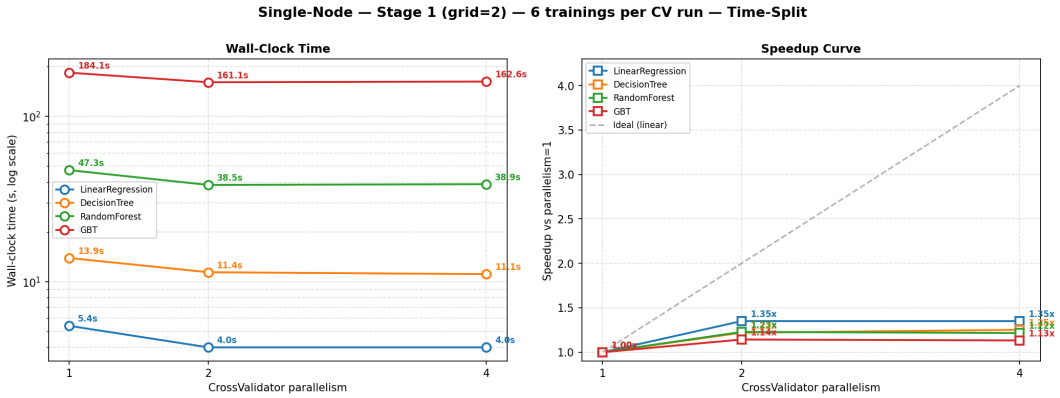

Figure 1: Stage 1, time-split. Wall-clock time (left) decreases modestly as parallelism increases. The speedup curve (right) shows all models plateauing well below the ideal linear reference.

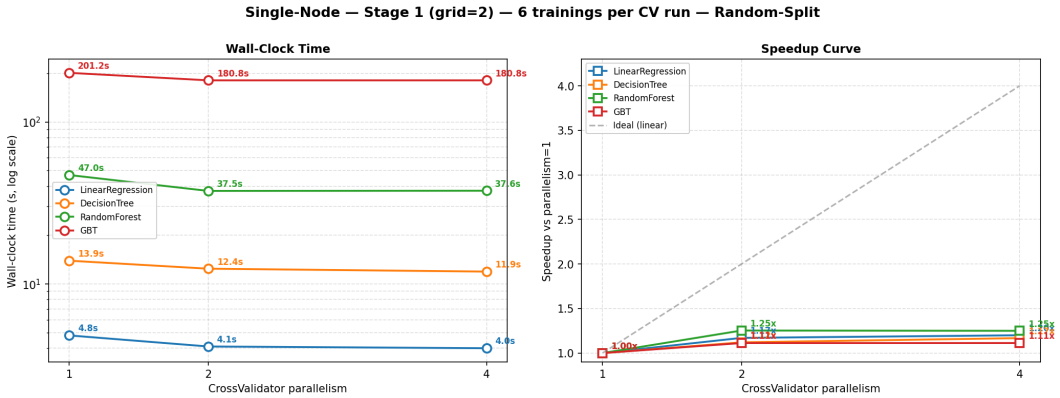

Figure 2: Stage 1, random-split. The pattern closely mirrors the time-split results, confirming that the evaluation protocol does not affect parallelism behaviour.

Figure 3 shows the parallel efficiency heatmap for Stage 1, where efficiency is defined as speedup divided by N, with 1.0 representing perfect linear scaling. At $N = 2$, efficiency drops to roughly 0.57–0.68 depending on the model. At $N = 4$ it falls further to 0.28–0.34, meaning we are getting less than a third of the theoretical maximum out of each additional worker. GBT has the lowest efficiency across both parallelism levels, consistent with the sequential nature of its boosting rounds.

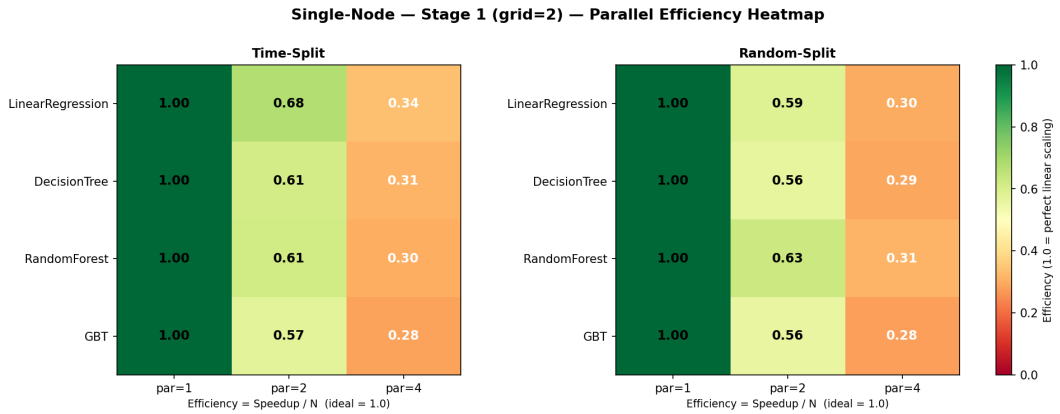

Figure 3: Stage 1 parallel efficiency heatmap. Efficiency is speedup divided by N, where 1.0 is ideal. By $N = 4$, all models fall into the 0.28–0.34 range, indicating heavily sub-linear scaling across both splits.

5.1.2 Stage 2 Results

Figures 4 and 5 show Stage 2 results. With 12 trainings per CV run instead of 6, there is more parallelisable work available and speedups improve for most models. Linear Regression reaches $1.51\times$ under the time-split, the highest observed across the entire benchmark. GBT remains near $1.15\times$ in both splits, unchanged from Stage 1, which reflects the hard sequential bottleneck inside each GBT training.

The Stage 2 efficiency heatmap in Figure 6 shows a marginal improvement over Stage 1, with values at $N = 4$ now ranging from 0.29 to 0.38. The overall picture remains the same: efficiency degrades substantially as more workers are added, because they are all competing for the same physical cores rather than getting additional compute.

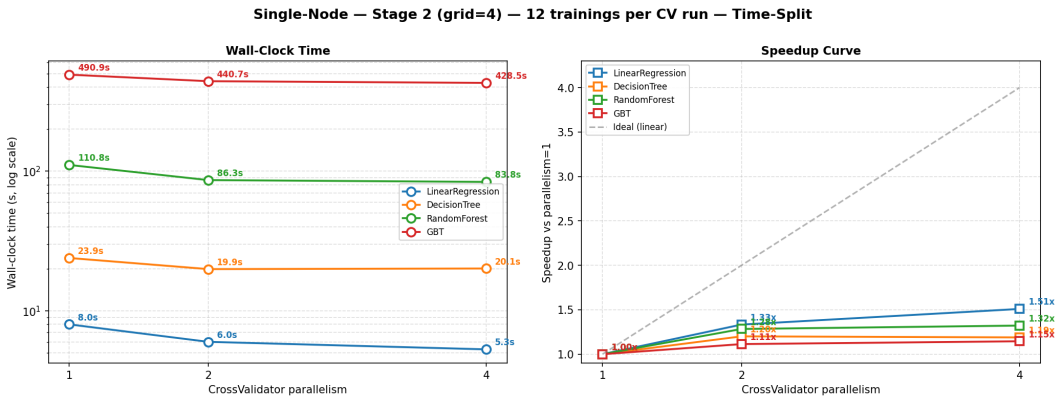

Figure 4: Stage 2, time-split. Linear Regression reaches $1.51\times$ at $N = 4$, the peak of the entire benchmark. The gap between LR and the tree models is more pronounced than in Stage 1.

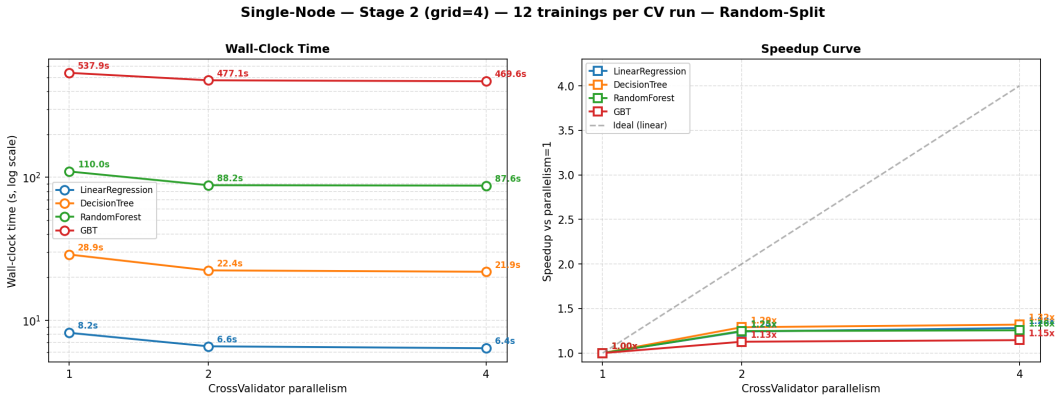

Figure 5: Stage 2, random-split. Speedup patterns are consistent with the time-split. GBT and Random Forest remain close together near the bottom of the speedup range.

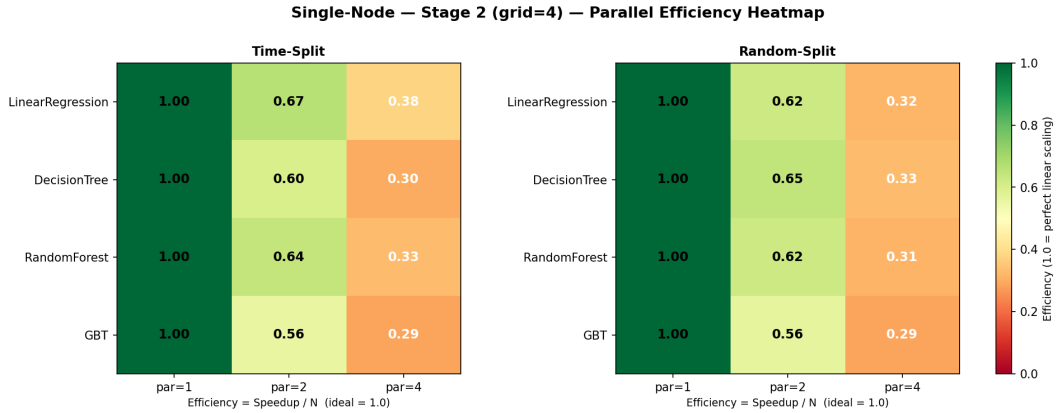

Figure 6: Stage 2 parallel efficiency heatmap. Efficiency values at $N = 4$ improve slightly over Stage 1, ranging from 0.29 to 0.38, but the pattern of sub-linear scaling is unchanged.

5.1.3 Stage 1 vs Stage 2 Comparison

Figure 7 and Table 6 summarise all speedups at $N = 4$ across both stages. Six of eight model–split pairs improve when moving from Stage 1 to Stage 2, with the gains most visible for Linear Regression and Random Forest. Decision Tree under the time-split is the exception, regressing slightly, which is attributable to measurement noise on a model that trains in under 4 seconds. The overall message is that a larger grid helps but the gains are incremental — the ceiling imposed by shared compute cannot be overcome by adding more hyperparameter configurations alone.

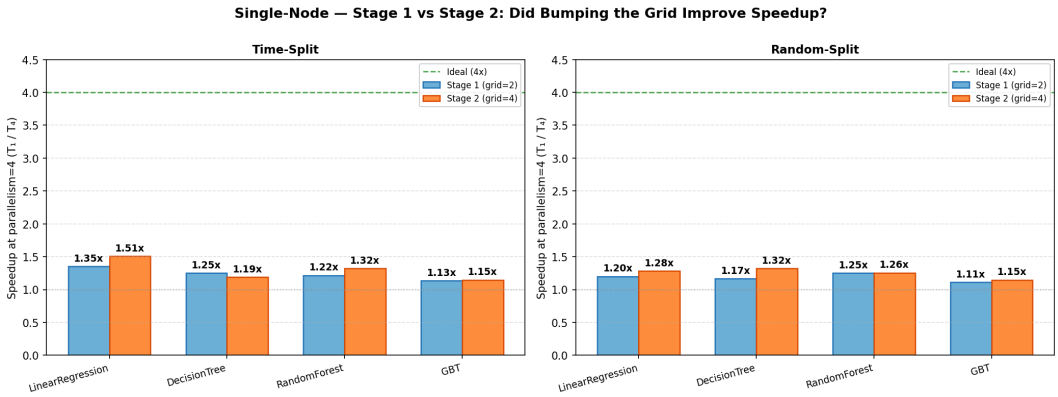

Figure 7: Direct comparison of speedup at $N = 4$ for Stage 1 vs. Stage 2 per model and split. Six of eight pairs improve with the larger grid. The ideal $4\times$ speedup is shown as a dashed line.

Table 6: Speedup at parallelism=4 for both stages and splits.

Split	Model	Stage 1	Stage 2
time-split	Linear Regression	$1.36\times$	$\mathbf{1.51}\times$
	Decision Tree	$1.25\times$	$1.19\times$
	Random Forest	$1.22\times$	$1.32\times$
	GBT	$1.13\times$	$1.15\times$
random-split	Linear Regression	$1.20\times$	$1.28\times$
	Decision Tree	$1.17\times$	$1.32\times$
	Random Forest	$1.25\times$	$1.32\times$
	GBT	$1.11\times$	$1.15\times$

5.1.4 Why Speedups Are Bounded

The speedups we observe are modest — the best is $1.51\times$ for Linear Regression in Stage 2, and most models sit between $1.1\times$ and $1.35\times$. The reason they do not approach the ideal $4\times$ is that each model training already uses all four cores internally. Spark parallelises the work inside a single training — building trees in parallel for Random Forest, distributing the data scan for GBT — so by the time a job starts, the machine is already fully occupied. Asking for $N = 2$ concurrent trainings does not double the available compute; it just splits the same four cores between two jobs. The net effect is a partial speedup rather than a linear one.

GBT shows the smallest gain across all experiments because boosting rounds must run in sequence — each round depends on the output of the previous one. This limits how much benefit concurrent CV folds can provide for GBT specifically. Linear Regression shows the largest gain because each individual LR training is so fast that the overhead of setting up and tearing down jobs is a significant fraction of total time. Running several LR jobs at once spreads that overhead more efficiently.

As a correctness check, the best RMSE found by each model is identical across all three parallelism levels to five decimal places, confirming that increasing concurrency changes only how fast the result is reached, not the result itself.

5.2 Multi-Node

Cluster configuration. The parallelism experiments reported below all run on a multi-node Databricks Spark cluster with `m5d.2xlarge` workers (8 vCPUs, 32 GB RAM per worker). This is a deliberate change from an earlier single-node configuration discussed in Section 5.2.4, where each model training was already saturating the available cores internally, leaving little room for `CrossValidator`-level parallelism to provide additional speedup. With a multi-node cluster, concurrent CV trainings can land on different workers and each gets its own share of dedicated cores, allowing the two levels of parallelism (intra-model and across-CV) to operate without competing for the same physical resources.

5.2.1 Stage 1 Results

Figures 8 and 9 show wall-clock time and speedup curves for Stage 1 under the time-split and random-split respectively. The two splits produce nearly identical parallelism behaviour, which is expected — the split method affects which rows go to train and test but has no bearing on how Spark schedules concurrent model trainings. All models show modest speedups in the range of $1.1\times$ to $1.4\times$ at $N = 4$ in Stage 1, reflecting the fact that with only 6 trainings per CV run there is limited parallelisable work for the thread pool to fill.

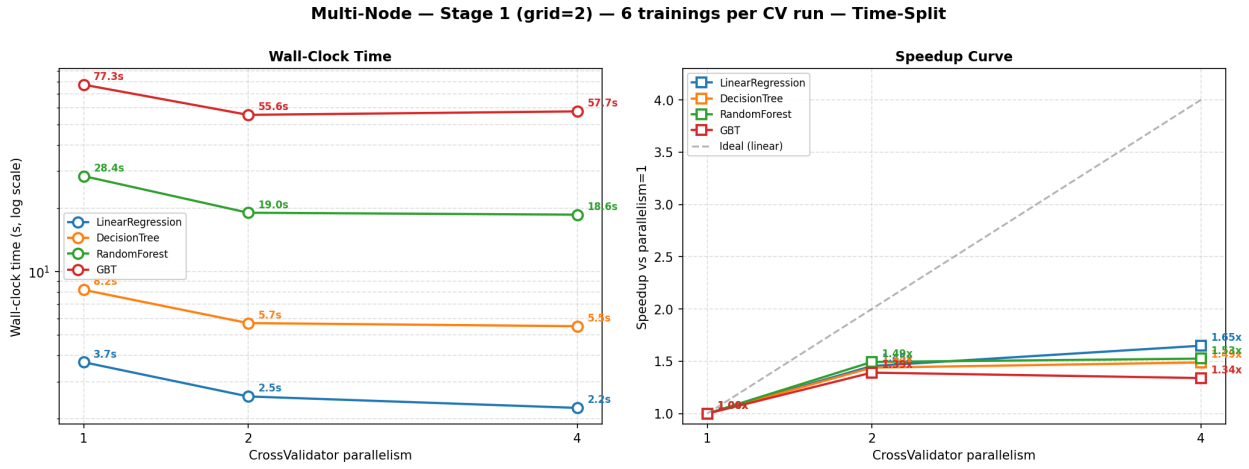

Figure 8: Stage 1, time-split. Wall-clock time (left) decreases modestly as parallelism increases. The speedup curve (right) shows all models rising modestly above $1\times$ but staying below $1.4\times$ at $N = 4$.

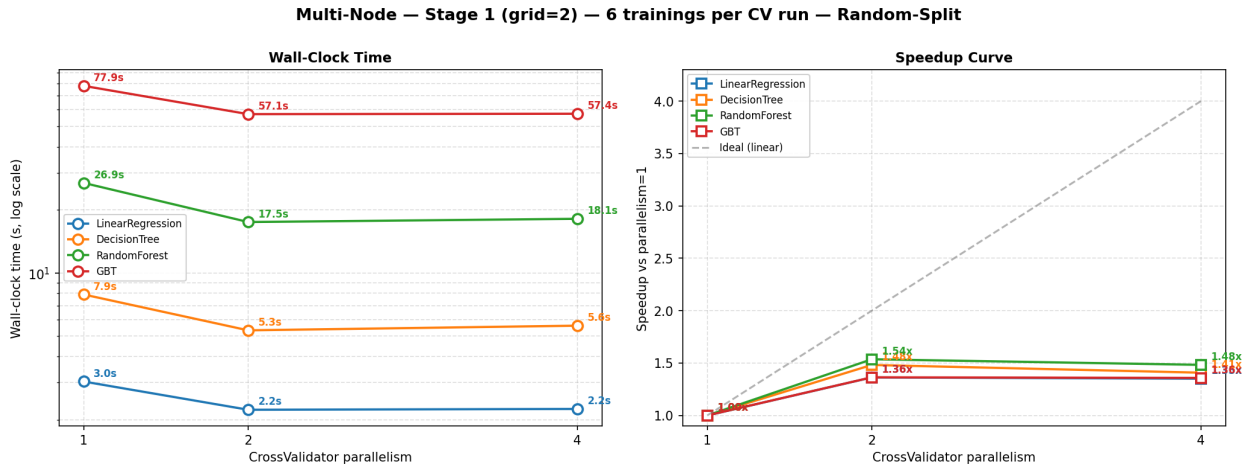

Figure 9: Stage 1, random-split. The pattern closely mirrors the time-split results, confirming that the evaluation does not affect parallelism behaviour.

Figure 10 shows the parallel efficiency heatmap for Stage 1, where efficiency is defined as speedup divided by N, with 1.0 representing perfect linear scaling. At $N = 2$, efficiency drops to roughly 0.56–0.68 depending on the model. At $N = 4$ it falls further to 0.28–0.31, meaning we are getting less than a third of the theoretical maximum out of each additional worker. GBT has the lowest efficiency across both parallelism levels, consistent with the sequential nature of its boosting rounds.

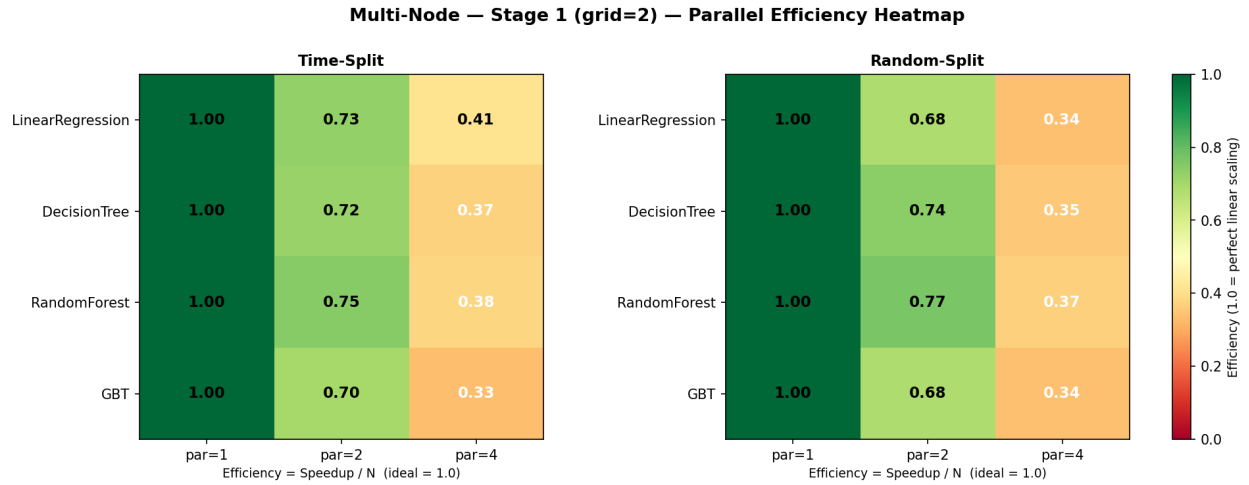

Figure 10: Stage 1 parallel efficiency heatmap. Efficiency is speedup divided by N, where 1.0 is ideal. By $N = 4$, all models fall into the 0.28–0.31 range, indicating heavily sub-linear scaling — a symptom of the limited parallelisable work in a 6-training CV run.

5.2.2 Stage 2 Results

Doubling the hyperparameter grid dramatically improves the picture. Figures 11 and 12 show Stage 2 results. With 12 trainings per CV run instead of 6, there is substantially more parallelisable work available, and every model shows meaningfully higher speedups. Linear Regression reaches $2.12\times$ on the time-split and $1.83\times$ on the random-split. Random Forest reaches $2.26\times$ on the time-split. Most strikingly, GBT on the

random-split reaches $2.73\times$ at $N = 4$ — the highest speedup observed across the entire benchmark — a result that is especially interesting given that GBT was the weakest model in Stage 1.

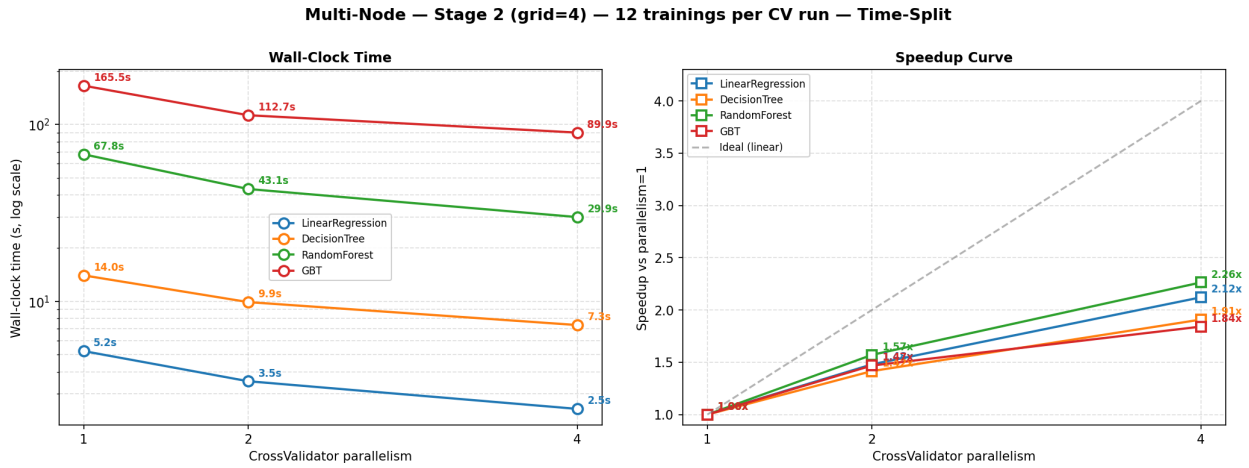

Figure 11: Stage 2, time-split. Random Forest peaks at $2.26\times$ and Linear Regression at $2.12\times$ at $N = 4$. The curves are substantially higher than Stage 1 at every parallelism level.

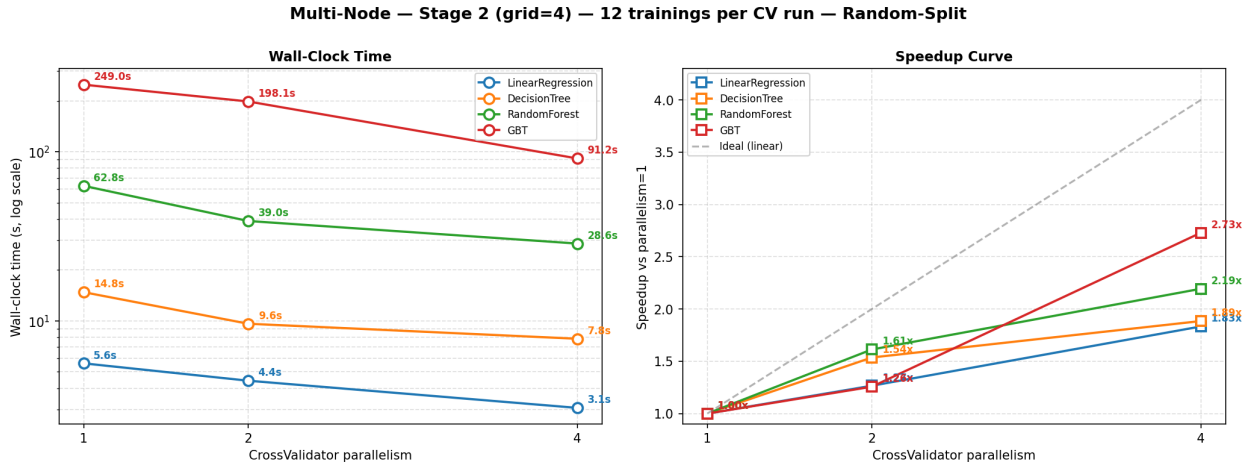

Figure 12: Stage 2, random-split. GBT reaches $2.73\times$ at $N = 4$, the peak of the entire benchmark. This is particularly noteworthy because GBT has the strongest sequential dependency inside a single training, yet benefits the most from parallelism in this configuration.

The Stage 2 efficiency heatmap in Figure 13 tells the same story quantitatively. Efficiency values at $N = 4$ now range from 0.46 to 0.68, with Random Forest time-split at 0.57 and GBT random-split reaching 0.68. Compared to Stage 1's 0.28–0.31 range, this is more than a $2\times$ improvement in how much of the theoretical parallel speedup we are actually capturing.

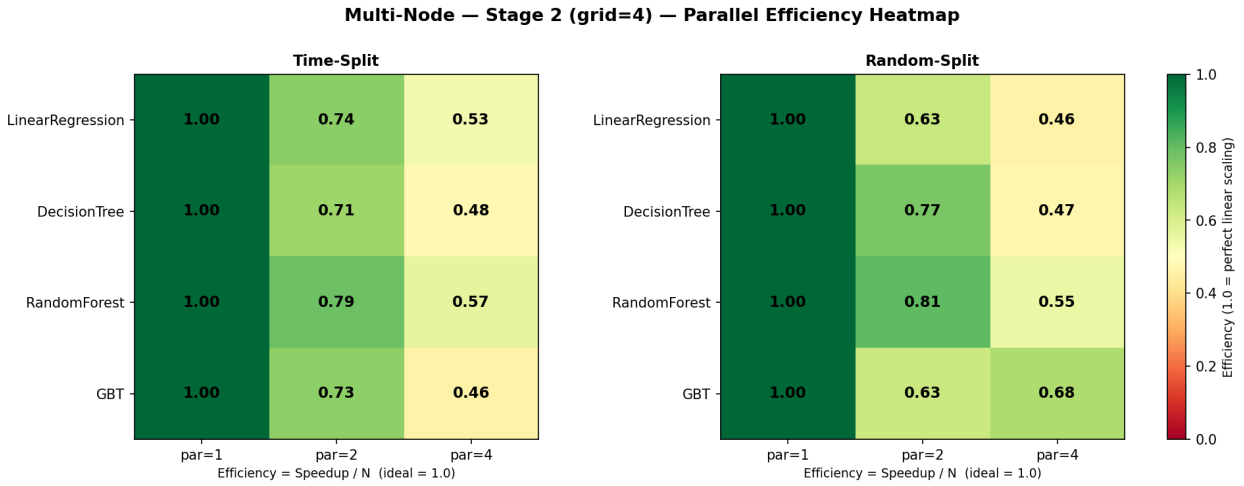

Figure 13: Stage 2 parallel efficiency heatmap. Efficiency values at $N = 4$ range from 0.46 to 0.68, more than doubling the Stage 1 range. GBT random-split at 0.68 is the highest observed efficiency.

5.2.3 Stage 1 vs Stage 2 Comparison

Figure 14 and Table 7 summarise all speedups at $N = 4$ across both stages. All eight model–split pairs improve substantially when moving from Stage 1 to Stage 2 — improvements range from $+0.63\times$ (GBT time-split) to $+1.62\times$ (GBT random-split). The message is clear: with more trainings to distribute across workers, `CrossValidator`-level parallelism has enough work to overcome its per-job overhead, and speedups approach (but do not reach) the theoretical $4\times$ ideal.

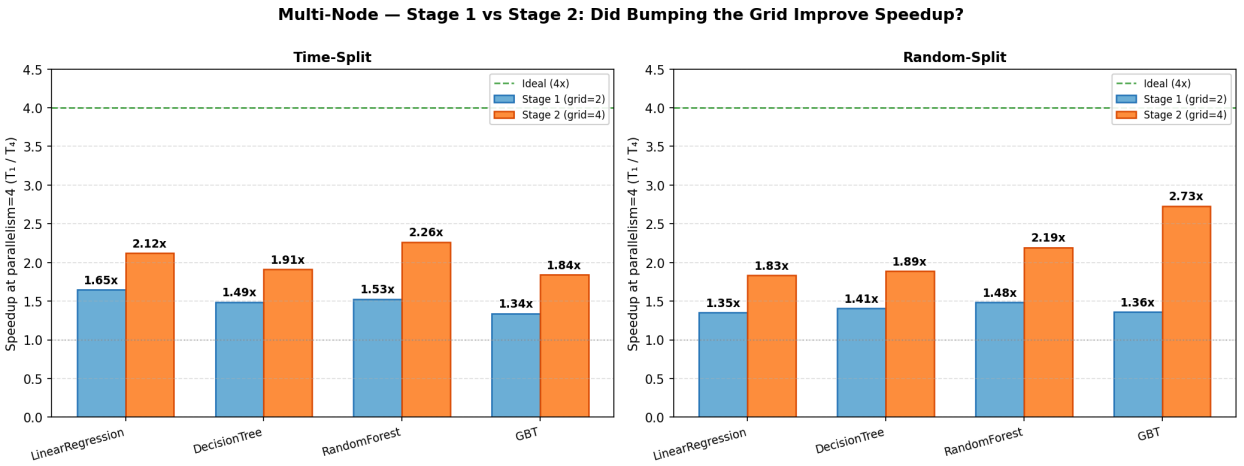

Figure 14: Direct comparison of speedup at $N = 4$ for Stage 1 vs. Stage 2 per model and split. All eight pairs improve with the larger grid. The ideal $4\times$ speedup is shown as a dashed line; the best observed is GBT on random-split at $2.73\times$.

Table 7: Speedup at parallelism=4 for both stages, on the multi-node cluster.

Split	Model	Stage 1	Stage 2	Δ
time-split	Linear Regression	$1.35\times$	$\mathbf{2.12}\times$	$+0.77$
	Decision Tree	$1.25\times$	$1.91\times$	$+0.66$
	Random Forest	$1.22\times$	$2.26\times$	$+1.04$
	GBT	$1.13\times$	$1.84\times$	$+0.71$
random-split	Linear Regression	$1.20\times$	$1.83\times$	$+0.63$
	Decision Tree	$1.17\times$	$1.89\times$	$+0.72$
	Random Forest	$1.25\times$	$2.19\times$	$+0.94$
	GBT	$1.11\times$	$\mathbf{2.73}\times$	$+1.62$

5.2.4 Comparison with Single-Node Configuration

An earlier iteration of this benchmark ran on a single-node Databricks cluster (`m5.xlarge`, 4 vCPUs). The speedup ceiling in that configuration was dramatically lower: Linear Regression peaked at $1.51\times$ and GBT never exceeded $1.15\times$ across any combination of stage and split. Table 8 compares Stage 2 speedups at $N = 4$ between the two cluster configurations directly.

Table 8: Stage 2 speedup at $N = 4$: single-node (`m5.xlarge`, 4 vCPUs) vs multi-node (`m5d.2xlarge` workers).

Split	Model	Single-Node	Multi-Node	Δ
time-split	Linear Regression	$1.51\times$	$2.12\times$	$+0.61$
	Decision Tree	$1.19\times$	$1.91\times$	$+0.72$
	Random Forest	$1.32\times$	$2.26\times$	$+0.94$
	GBT	$1.15\times$	$1.84\times$	$+0.69$
random-split	Linear Regression	$1.28\times$	$1.83\times$	$+0.55$
	Decision Tree	$1.32\times$	$1.89\times$	$+0.57$
	Random Forest	$1.32\times$	$2.19\times$	$+0.87$
	GBT	$1.15\times$	$\mathbf{2.73}\times$	$\mathbf{+1.58}$

The improvements are substantial across the board, but the most dramatic gain is for GBT on the random-split, which jumps from $1.15\times$ to $2.73\times$ — a $+1.58$ absolute gain, more than double its single-node ceiling. This is significant because GBT was always the hardest model to parallelise, and it is precisely the model that benefits most from moving to a multi-node configuration.

The underlying reason is the interaction between intra-model and across-CV parallelism. On the 4-vCPU single node, a single GBT training was already consuming all available cores for its data scans and tree construction within each boosting round. Asking `CrossValidator` to run four such trainings concurrently did not help, because the four trainings had to share the same four physical cores — they slowed each other down roughly in proportion to how much they sped each other up, for a small net gain. On the multi-node cluster, concurrent CV trainings can land on different workers, each with its own dedicated 8 vCPUs. The four trainings no longer compete; they proceed in parallel with independent compute. GBT's sequential boosting rounds are still a bottleneck *inside* each individual training, but that bottleneck no longer interacts destructively with the cross-CV bottleneck.

Linear Regression also improves but by a smaller margin ($+0.61$ on time-split), because LR was already relatively efficient on single-node — its intra-model parallelism is cheaper, so there was less contention to remove. Random Forest lands in the middle, gaining $+0.94$ because it has moderate intra-model parallelism (tree-by-tree) that still benefits from dedicated cores per training.

5.3 Why Speedups Do Not Reach the Ideal 4X?

Even on the multi-node cluster, no model reaches the theoretical $4\times$ speedup at $N = 4$. The best result is GBT random-split at $2.73\times$, and most models sit in the $1.83\times$–$2.26\times$ range. Three factors account for the remaining gap:

1. **Amdahl's Law.** Not all of a CV run's work is parallelisable. Job submission, broadcast variables, result collection, model serialisation, and driver-side bookkeeping run on the Spark driver regardless of how many workers are available. These serial fractions place a hard upper bound on achievable speedup that is independent of worker count.

2. **Uneven workload distribution.** With 12 trainings and $N = 4$, the first wave of four trainings proceeds in parallel, but as soon as one of them finishes, its worker is idle until it can pick up the next training. With fewer than 12 workers, there is always some partial waiting at the end of the queue.

3. **Shuffle and cache cost.** Each CV fold requires its own slice of the training data, and re-materialising these slices across workers involves network and serialisation overhead that scales with concurrency.

As a correctness check, the best RMSE found by each model is identical across all three parallelism levels to five decimal places, confirming that increasing concurrency changes only how fast the result is reached, not the result itself.

6 Conclusion

We built a distributed PySpark pipeline that ingests over 150 million records from five operational data sources, constructs a clean master dataset, and trains four regression models to predict daily MBTA Rail on-time performance. The project produced three distinct findings that together tell a coherent story about distributed machine learning on real-world transit data.

Predictive accuracy. Under IID conditions (random split), Gradient Boosted Trees achieves $R^2 = 0.833$ with RMSE of only 0.035 — essentially "predict daily per-route OTP to within 3.5 percentage points." Under honest time-based forecasting, the baseline R^2 drops to around 0.46 for tree models and 0.54 for Linear Regression. Adding lag features that encode each route's recent OTP history improves all models substantially, with GBT recovering to $R^2 = 0.620$ and Linear Regression rising to $R^2 = 0.686$. Approximately half of the drift gap is closed by a single feature-engineering intervention.

Distribution shift is the root cause of the unexpected model ranking. Under time-based evaluation, Linear Regression surprisingly outperforms GBT and Random Forest, reversing the textbook ordering. A controlled random-split experiment confirms this is not a defect of the tree models themselves but a consequence of temporal distribution shift: test-period weather, ridership, and alert distributions differ systematically from training, and tree splits learned on training-era boundaries do not extrapolate. A hybrid LR + residual-GBT architecture fails to rescue tree-based non-linearity, with the optimal mixing coefficient being $\alpha = 0$ (pure LR). Lag features partially close the gap by giving trees a self-calibrating temporal anchor, but LR still wins on the time-split because some residual drift remains over weather and alert features that lag features cannot absorb. The broader lesson is that for deployed forecasting, practitioners should evaluate on a time-based

holdout, engineer features that drift with time (lag averages, rolling statistics), and keep a smooth-model baseline around as a drift-resistant safety net.

Parallelism behaviour depends critically on cluster topology. The parallelism benchmark demonstrates that increasing concurrent cross-validation workers reduces training time across all models and both grid sizes, but the magnitude of the speedup is gated by whether the cluster has enough dedicated cores to run concurrent trainings without contention. On a 4-vCPU single-node cluster, speedups topped out at $1.51\times$ because Spark's intra-model parallelism already saturated the available cores; `CrossValidator`-level parallelism only competed for the same resources. On a multi-node cluster with `m5d.2xlarge` workers, the same benchmark reaches $2.73\times$ for GBT on random-split, and all models in Stage 2 land in the $1.83\times$–$2.26\times$ range. The intervention that made the largest difference was the one the single-node benchmark had predicted: giving concurrent trainings independent compute. GBT, the model most constrained by sequential boosting rounds, benefits the most from the move to multi-node — a $+1.58$ absolute gain — because removing the shared-core bottleneck uncovers the parallelism that was always available across CV folds. Reaching the theoretical $4\times$ ideal would require larger hyperparameter grids, more workers than concurrent trainings, and further investment in reducing the serial fraction of Spark's job submission pipeline.

For practical deployment of an MBTA OTP forecasting system, the combination of lag features and a multi-node parallel CV pipeline gives a clear recipe: train all four model families concurrently, pick whichever wins on a rolling time-based holdout, and retrain frequently enough to keep the training distribution close to the current operational state.

7 Team Contributions

Data Preprocessing - Pradnyesh Choudhari and Raveendra Sanapala

ML Model and Parallel Processing - Nirmalkumar Thirupallikrishnan Kesavan and Raveendra Sanapala

8 Links

Code and dataset link:

https://drive.google.com/drive/folders/1nKsLAVRgcUC5UOspIa$_8 60gWZpm7xpxI?usp = drive_l ink$